Building a Working SIEM Lab

Deploying a Security Information and Event Management (SIEM) Solution for Security Monitoring and Investigation.

Environment: Virtual Lab Environment.

Analyst: Wontumi

30th July, 2026.

Table of Contents

Executive Summary....3
Scope and Objectives....4
Methodology....6
- Lab Architecture....6
- Log Flow within the Environment....8
- Environment Setup....8
  - Hardware Environment....8
  - Software Environment....9
- Implementation Phases....9
  - Planning....10
  - Overview....10
  - Project Requirements....10
  - Project Goals....10
  - Wazuh Installation and Initial Configuration....11
  - Overview....11
  - Installing the Wazuh Platform....11
  - Verifying the Dashboard....11
  - Windows Endpoint Integration....13
  - Windows Log Sources....16
  - Ubuntu Endpoint Integration....16
  - Ubuntu Log Sources....18
  - pfSense Firewall Log Integration....18

Findings....20
- SIEM Dashboard Development....20
  - Dashboard 1 – Authentication Activity Dashboard....20
  - Objective....20
  - Dashboard 2 – Endpoint Process Activity Dashboard....21
  - Objective....21
  - Dashboard 3 – Network Traffic Overview Dashboard....22
  - Objective....22
- Correlation Rules....23
  - Rule 1 – Failed Login Threshold....23
  - Rule 2 – Suspicious PowerShell Execution....24

Gaps and Challenges....26
Recommendations and Next Steps....27

Executive Summary

Security Information and Event Management (SIEM) systems help organizations collect and monitor security information from different computers and network devices in one central location. In this project, I designed and built a fully functional SIEM laboratory using Wazuh within a virtual environment. The lab included a Kali Linux system running the Wazuh platform, a Windows 11 computer with Sysmon, an Ubuntu Linux computer, and a pfSense firewall, all configured to send security logs to the SIEM.

To test the environment, I performed normal and security-related activities such as creating files, running applications, logging in through SSH, using administrative commands, and generating network traffic. These activities produced security events that were collected, stored, and displayed in the Wazuh Dashboard for monitoring and investigation.

The completed lab demonstrated how security events from different systems can be brought together in one place, making it easier to detect unusual activity and investigate potential security incidents. This project also provided practical experience in deploying a SIEM, configuring log collection, monitoring endpoints and firewall activity, and analyzing security events in a simulated Security Operations Center (SOC) environment.

Scope and Objectives

This project focuses on the deployment of a Security Information and Event Management (SIEM) laboratory using Wazuh within a virtualized environment. The implementation includes the integration of Windows, Ubuntu Linux, and pfSense log sources to demonstrate centralized log collection and security monitoring. While the environment simulates a small enterprise network, it does not implement high availability, clustering, or advanced threat intelligence integration. The primary emphasis is on log ingestion, endpoint monitoring, firewall logging, event analysis, and practical SOC workflows suitable for educational and demonstration purposes.

The main goal of this project was to create and set up a fully functional Security Information and Event Management (SIEM) lab that could gather, monitor, and analyze security events from various systems in a virtual setting. I aimed to replicate a real-world Security Operations Center (SOC) by bringing together endpoint monitoring, centralized log collection, and network security monitoring into one cohesive platform. To make this happen, I laid out the following objectives:

- Deploy the Wazuh SIEM platform as our go-to solution for log management and security monitoring.
- Set up a Windows 11 endpoint with the Wazuh Agent and Microsoft Sysmon to capture detailed security events from Windows.
- Configure an Ubuntu Linux endpoint with the Wazuh Agent to keep an eye on Linux authentication, system, and user activity logs.
- Install a pfSense firewall to mimic network perimeter security and send firewall and system logs to the SIEM using the Syslog protocol.
- Design a secure virtual network architecture that allows communication between the Wazuh Manager, endpoints, and firewall while ensuring Internet access for software updates and activity simulation.
- Generate realistic activities on endpoints and networks, such as process execution, PowerShell commands, SSH authentication, privileged command execution, software installations, and web browsing, to create meaningful security events.
- Confirm that logs from all configured sources are successfully collected, indexed, and visualized in the Wazuh Dashboard.

- Showcase how centralized logging and event correlation can enhance visibility into endpoint and network activities.
- Create technical documentation that outlines the deployment process, network architecture, implementation phases, challenges faced, and lessons learned throughout the project.

By completing these objectives, the project provides practical experience in SIEM deployment, log management, endpoint monitoring, network security monitoring, and security event investigation. These skills closely align with the responsibilities of Security Operations Center (SOC) analysts responsible for detecting, investigating, and responding to security incidents in enterprise environments.

Methodology

Lab Architecture

The goal of this project was to create and set up a fully functional Security Information and Event Management (SIEM) lab that could gather, centralize, and analyze security logs from various sources. Before diving into the deployment, I crafted a network architecture that mimics a small enterprise setup, all while keeping it straightforward for testing, troubleshooting, and security monitoring.

In this lab, I have a Kali Linux host running the Wazuh Manager and Dashboard, along with two monitored endpoints, one running Windows 11 and the other Ubuntu Linux. There's also a pfSense firewall in the mix, which handles traffic routing and generates firewall logs. To ensure smooth communication between all the virtual machines and the host system, I set up a VirtualBox Host-Only Network. Meanwhile, the virtual machines still have Internet access through Network Address Translation (NAT). This setup allows the endpoints to securely connect with the Wazuh Manager while also reaching out to external resources needed for software installations, updates, and activity simulations.

I made sure the architecture kept internal management traffic separate from Internet traffic. All security logs from the Windows endpoint, Ubuntu endpoint, and pfSense firewall are sent to the Wazuh Manager, where they’re centralized, indexed, and visualized via the Wazuh Dashboard for effective monitoring and investigation.

The completed laboratory architecture is illustrated in **Figure 1**.

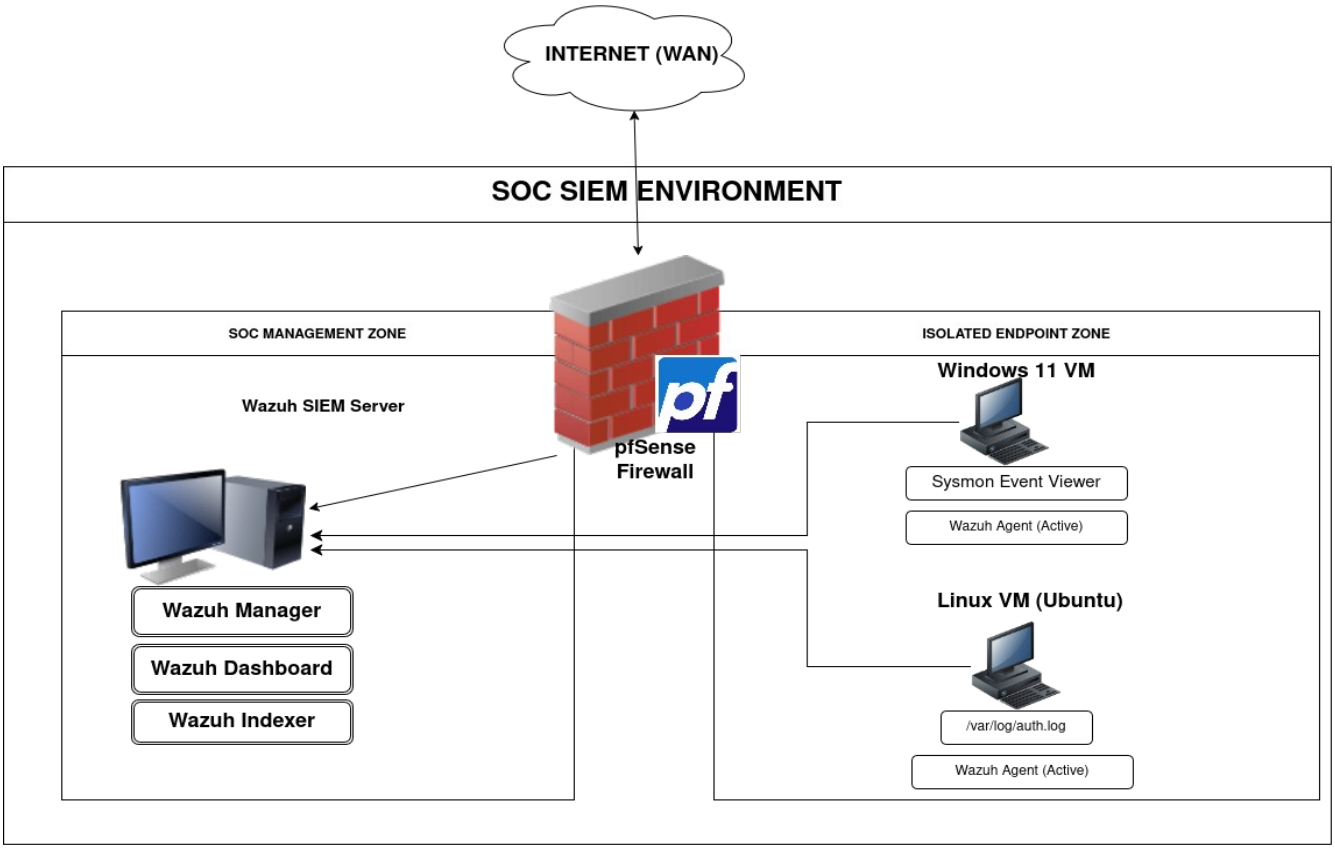


Figure 1: SOC Lab Architecture

Table 3.1 – Laboratory Components

Component	Purpose
Kali Linux (Host)	Hosted the Wazuh Manager, Indexer, and Dashboard, serving as the centralized SIEM platform responsible for collecting, storing, and analyzing security logs.
Windows 11 Virtual Machine	Simulated a Windows endpoint. The system was configured with Microsoft Sysmon and the Wazuh Agent to generate and forward detailed Windows security events.
Ubuntu Linux Virtual Machine	Simulated a Linux endpoint. The Wazuh Agent monitored authentication logs, SSH activity, privileged command execution, package installation, and other Linux system events.
pfSense Firewall	Served as the network firewall, controlling

	communication between the internal network and external networks while forwarding firewall and system logs to the SIEM through Syslog.
VirtualBox Host-Only Network	Provided secure internal communication between the Wazuh Manager, endpoints, and pfSense without exposing management traffic to the external network.
NAT Network	Provided Internet connectivity for software installation, updates, and activity generation while maintaining network isolation for the laboratory environment.

Log Flow within the Environment

To get a clearer picture of how security events were handled, I set up an environment where logs from each monitored device were sent to a central hub for analysis. The Wazuh Agents on both Windows and Ubuntu collected endpoint logs and securely forwarded them to the Wazuh Manager. Meanwhile, firewall and system events from pfSense were sent over to the Kali host using the Syslog protocol before being brought into the Wazuh platform for analysis and visualization.

This centralized logging setup made it possible for me to keep an eye on activities happening across various systems from one dashboard, which really enhanced visibility and made security investigations a lot easier.

Environment Setup

Hardware Environment

The project was carried out on a single physical computer that had Kali Linux as its host operating system. I utilized virtualization technology to set up isolated virtual machines, which “mimicked enterprise endpoints and network infrastructure”. By running all the components on one physical device, we kept the environment portable while ensuring there were enough resources available for testing the SIEM functionality and collecting logs.

Table 4.1 – Host System Specifications

Hardware Component	Specification
Host Operating System	Kali Linux
Virtualization Platform	Oracle VirtualBox

Installed Memory (RAM)	*8 GB*
Storage	*256 GB SSD*
Network Connection	Wireless Network

Software Environment

To set up the SIEM laboratory, I rolled out a variety of software components that together mimic a small enterprise environment. Each application was carefully chosen to serve a particular purpose, covering everything from endpoint monitoring and firewall logging to centralized log management and event analysis.

Table 4.2 – Software Components

Software	Purpose
Kali Linux(host)	Hosted the Wazuh platform and served as the primary management system.
Wazuh Manager	Collected, analyzed, and correlated security events from all monitored devices.
Wazuh Dashboard	Provided dashboards, alert visualization, log searching, and investigation capabilities.
Windows 11	Simulated a Windows workstation for endpoint monitoring.
Microsoft Sysmon	Generated enhanced Windows telemetry, including process creation, network connections, and file activity.
Ubuntu Linux	Simulated a Linux endpoint for authentication and system monitoring.
pfSense	Simulated the enterprise firewall and generated network security events.
Oracle VirtualBox	Hosted the virtual machines and provided the networking infrastructure required for the laboratory.
draw.io	Used to design the network architecture diagram included in this report.

Implementation Phases

In this chapter, I’ll walk you through how we practically set up the SIEM laboratory. We kicked things off with careful planning and prepping the environment, then moved on to

installing and configuring the Wazuh platform along with the endpoint systems. By breaking the project into phases, I was able to test each component on its own before bringing everything together into a fully functional security monitoring setup.

Planning

Overview

Before deploying the SIEM environment, I developed a clear implementation plan to ensure that all required components could communicate effectively and that the project objectives could be achieved within the available time. The planning phase focused on identifying the technologies required, designing the laboratory architecture, and defining the log sources that would be integrated into the Wazuh platform.

Rather than deploying the environment immediately, I first identified the expected data flow between the endpoints, firewall, and SIEM platform. This helped minimize configuration errors during deployment and simplified troubleshooting as additional components were introduced into the environment.

Project Requirements

To get the laboratory up and running smoothly, I pinpointed a few key technical requirements:

- A virtualization platform that can handle multiple virtual machines.
- A dedicated SIEM platform for gathering and analyzing logs in one place.
- A Windows endpoint that can generate in-depth security telemetry.
- A Linux endpoint to keep an eye on authentication and system events.
- A firewall that can produce network security logs.
- Reliable network connectivity linking all the lab components.
- Internet access for installing software, updating packages, and simulating activities.

These requirements laid the groundwork for the lab, making sure the environment closely mimicked a small enterprise network.

Project Goals

The primary goals established during the planning phase were:

- Deploy a functional Wazuh SIEM platform.
- Collect security logs from Windows, Ubuntu Linux, and pfSense.
- Centralize all collected logs within a single monitoring platform.
- Simulate realistic endpoint and network activities for security monitoring.
- Validate successful log collection from each configured source.
- Build a dashboard and write correlation rules.

- Produce a documented SIEM laboratory suitable for demonstrating practical SOC skills.

Wazuh Installation and Initial Configuration

Overview

After completing the planning stage, I proceeded with the deployment of the Wazuh SIEM platform. Since Wazuh serves as the central component responsible for collecting, indexing, and analyzing security events, I completed its installation before configuring any monitored endpoints. This approach ensured that log sources could be validated immediately after they were integrated into the environment.

The Wazuh platform was deployed on the Kali Linux host operating system, which served as the centralized management server throughout the project.

Installing the Wazuh Platform

I installed the Wazuh platform using the official installation procedure provided by the Wazuh documentation.

Wazuh installation command:

```
“curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh && sudo bash ./wazuh-install.sh -a”
```

The installation included the core services required for a complete SIEM deployment, namely:

- Wazuh Manager
- Wazuh Indexer
- Wazuh Dashboard

Once the installation was completed successfully, I verified that all required services were running correctly before proceeding with endpoint integration.

Verifying the Dashboard

Once the installation was complete, I hopped onto the Wazuh Dashboard via the web interface to check if everything was running smoothly. Being able to access the dashboard successfully was a good sign that the management server, indexer, and visualization components were all working as they should.

The dashboard acts as the primary interface for monitoring security alerts, searching through logs, managing agents, and digging into security events gathered from across the environment.

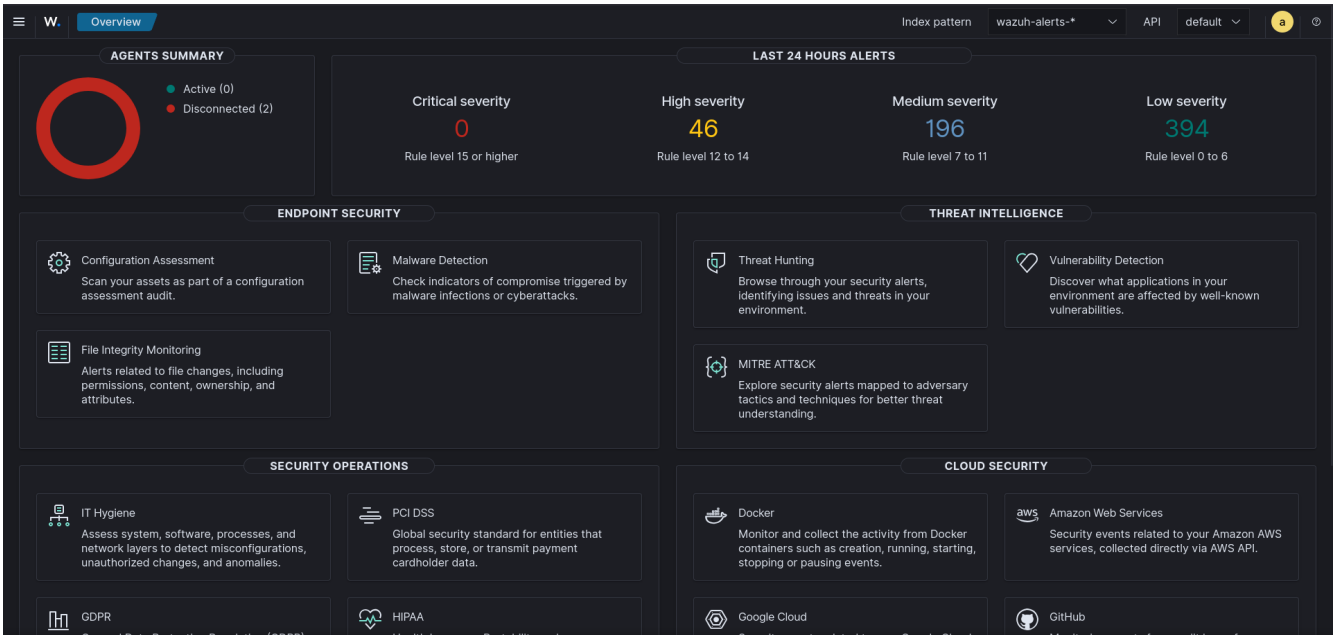


Figure 2: Wazuh Dashboard Home Page

After confirming dashboard accessibility, I verified that the Wazuh Manager service was operational and capable of accepting endpoint registrations. This verification was necessary before deploying Wazuh Agents to the Windows and Ubuntu systems.

Successful manager verification confirmed that the SIEM infrastructure was prepared to receive logs from multiple monitored endpoints.

Now that the Wazuh Manager is up and running, I moved on to registering the endpoint agents. This registration process set up a secure line of communication between each endpoint and the Wazuh Manager, enabling the forwarding of endpoint logs for centralized analysis.

To start, I checked the communication with just one endpoint before slowly bringing more systems online. This step-by-step method made troubleshooting easier and helped me ensure that everything was connected properly before expanding the setup.

I confirmed the successful registration of agents through the Wazuh Dashboard, where I could see the connected endpoints listed with an Active status.

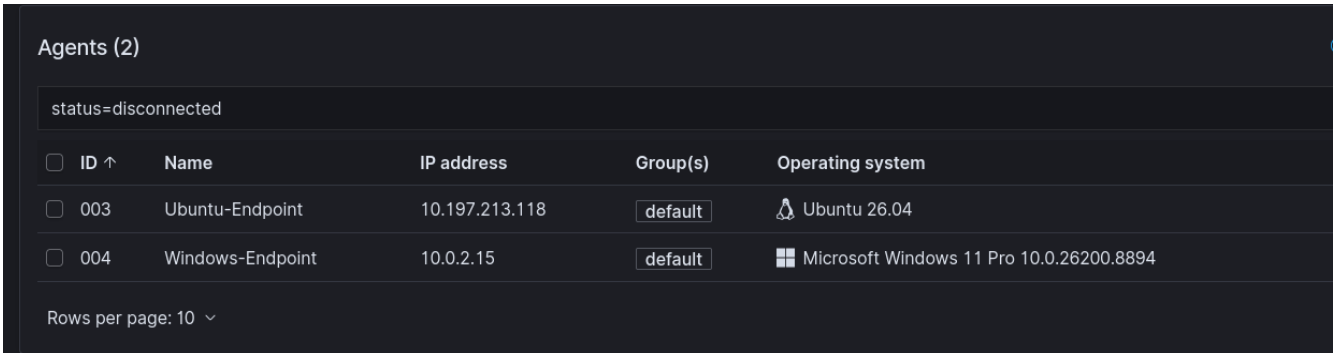


Figure 3: Registered Wazuh Agents

Windows Endpoint Integration

After successfully rolling out the Wazuh platform, I brought a Windows 11 virtual machine into the SIEM environment to act as the main Windows endpoint. The goal was to gather detailed Windows security events and send them over to the Wazuh Manager for centralized monitoring and analysis.

I kicked things off by installing the Wazuh Agent on the Windows 11 system and registering it with the Wazuh Manager. Once the agent was up and running, I checked the Wazuh Dashboard and saw that the endpoint was marked as Active, which confirmed that secure communication was established between the endpoint and the SIEM platform.

To boost the visibility of Windows events, I went ahead and installed Microsoft Sysmon (System Monitor) from the Sysinternals suite. Sysmon enhances the default Windows logging by capturing detailed system activities like process creation, network connections, driver loading, file creation, and registry changes. This extra telemetry gives us a clearer picture of endpoint behavior and is a go-to tool for Security Operations Centers (SOCs) when it comes to threat detection and investigation.

After getting Sysmon set up, I configured the Wazuh Agent to collect Sysmon event logs from the Windows Event Log channel. This setup allowed Sysmon events to be automatically forwarded to the Wazuh Manager, right alongside the standard Windows security logs.

To validate the configuration, I generated several Windows activities, including:

- Running Command Prompt.
- I attacked Windows with Metasploit on Kali.
- Executed a malicious “shell.exe” file.
- Created a backdoor user and Escalated privileges.

- Executing administrative commands.
- Installed “mimikatz” and deleted a user.

These activities successfully generated security events, which were collected and displayed in the Wazuh Dashboard, confirming that the Windows endpoint had been fully integrated into the SIEM environment.

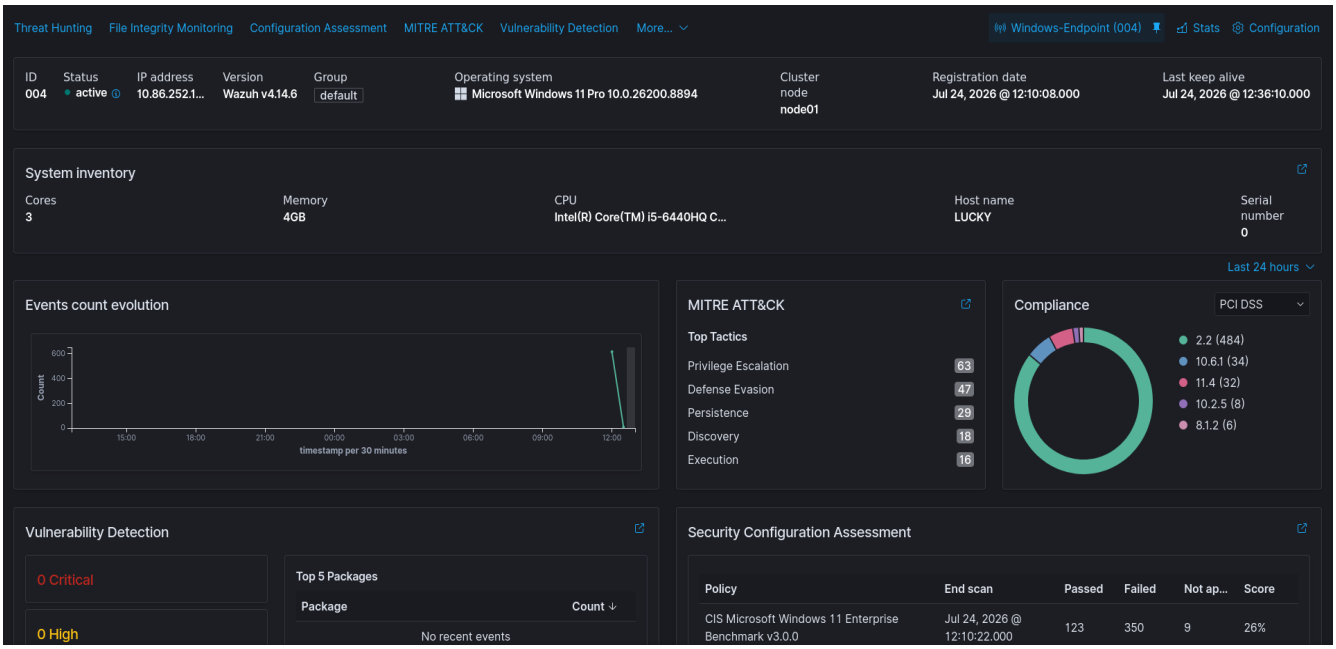


Figure 4: Windows 11 endpoint registered in the Wazuh Dashboard.

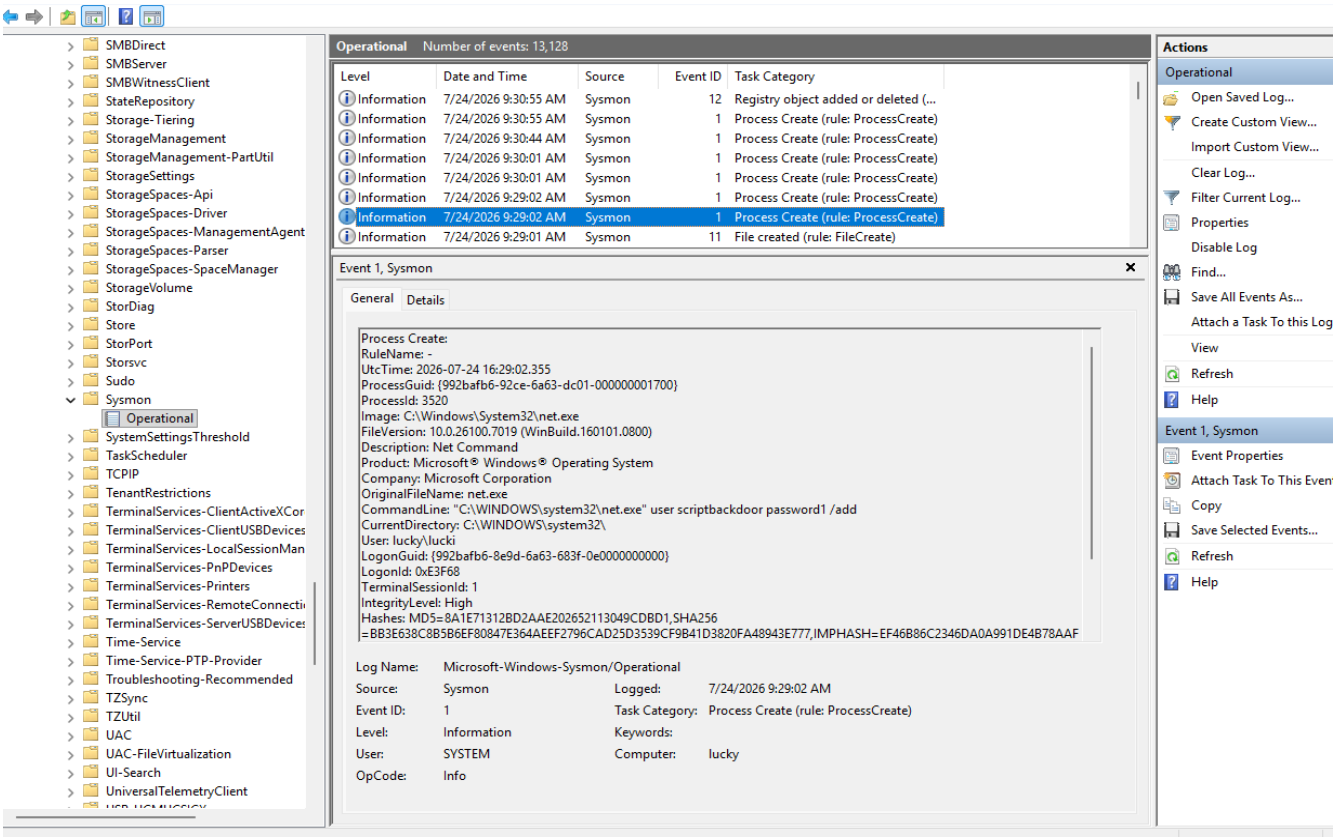


Figure 5: Sysmon events displaying Windows logs.

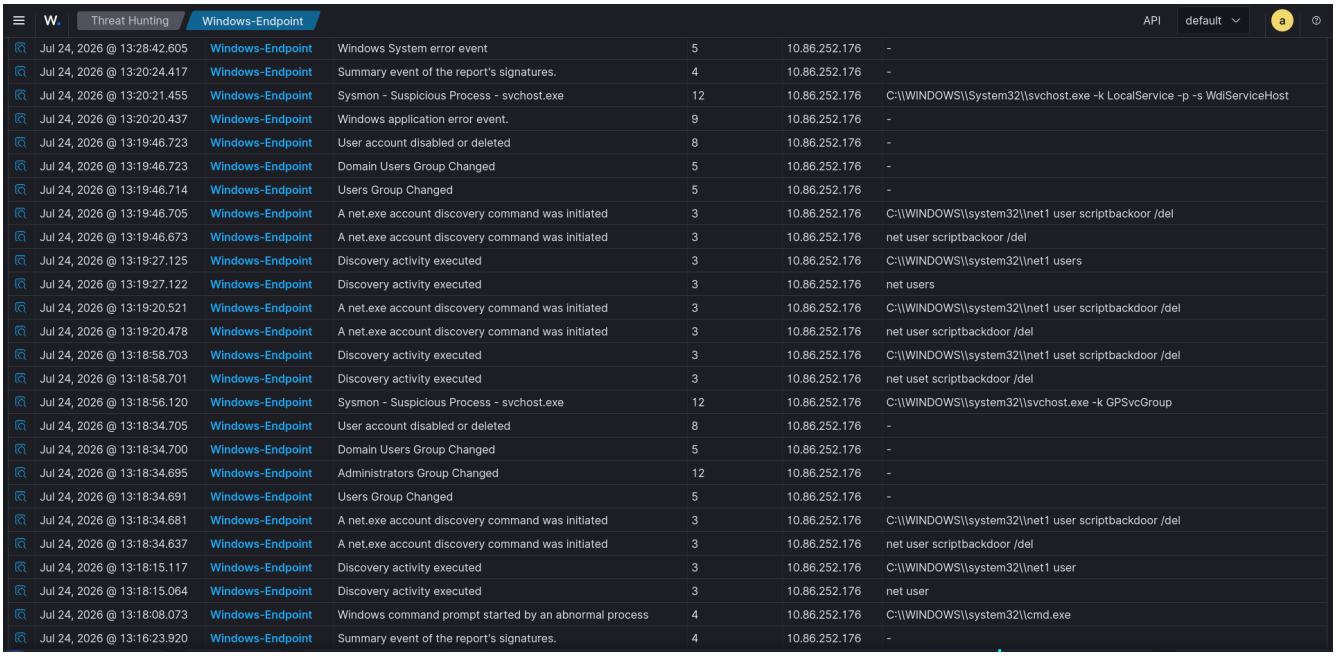


Figure 6: Sysmon events displayed in the Wazuh Dashboard.

Windows Log Sources

Log Source	Description	On-boarding Method
Windows Security Log	User authentication, logon, and system security events.	Collected automatically through the Wazuh Agent.
Microsoft Sysmon	Detailed process, network, file, and system activity events.	Installed Sysmon and configured the Wazuh Agent to monitor the Sysmon event channel.
Command Logs	Command activity.	Collected through Windows Event Logs by the Wazuh Agent.

Ubuntu Endpoint Integration

To extend monitoring beyond the Windows environment, I integrated an Ubuntu Linux virtual machine into the SIEM laboratory. This endpoint was used to generate Linux authentication and system events, allowing the Wazuh platform to monitor activity across multiple operating systems.

I installed the Wazuh Agent on the Ubuntu virtual machine and configured it to communicate with the Wazuh Manager. After registration, I verified that the Ubuntu endpoint appeared as Active in the Wazuh Dashboard, confirming successful connectivity.

Unlike Windows, Linux records most security-related activities in system log files. The Wazuh Agent automatically monitored these logs and forwarded relevant events to the Wazuh Manager for analysis.

To validate the integration, I generated several Linux activities, including:

- Successful and failed SSH login attempts.
- Execution of privileged commands using `sudo`.
- Creation and modification of files.
- Installation of software packages using the `apt` package manager.
- General command-line activity.

These activities produced authentication and system events that were successfully ingested into the Wazuh platform, demonstrating that Linux logs were being collected and analyzed correctly.

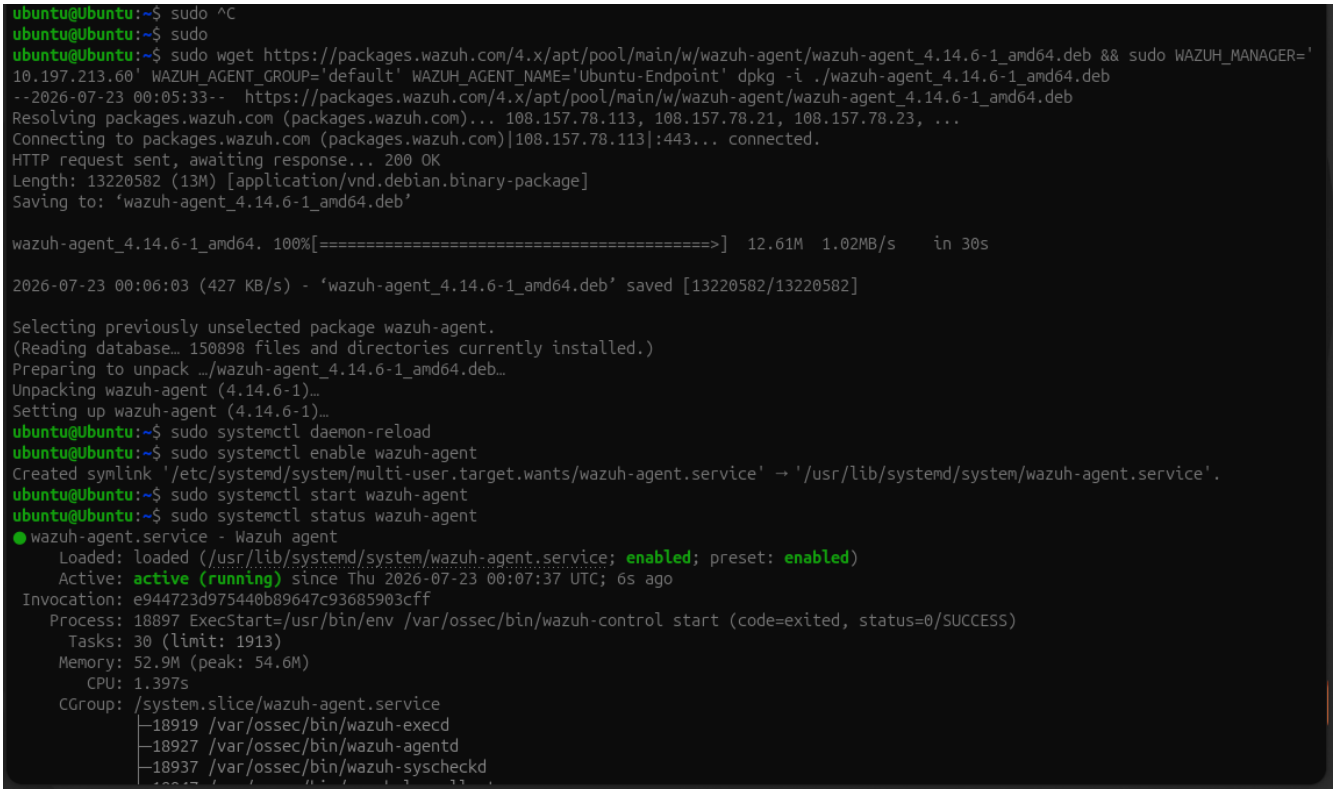


Figure 7: Wazuh-agent actively running on Ubuntu endpoint

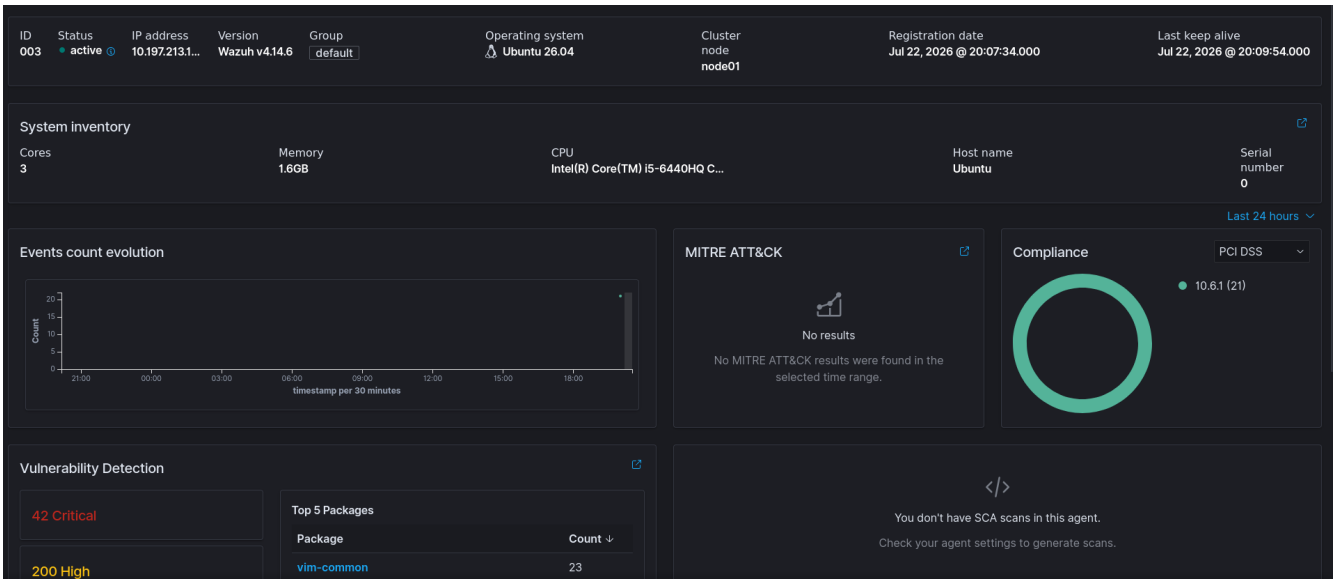
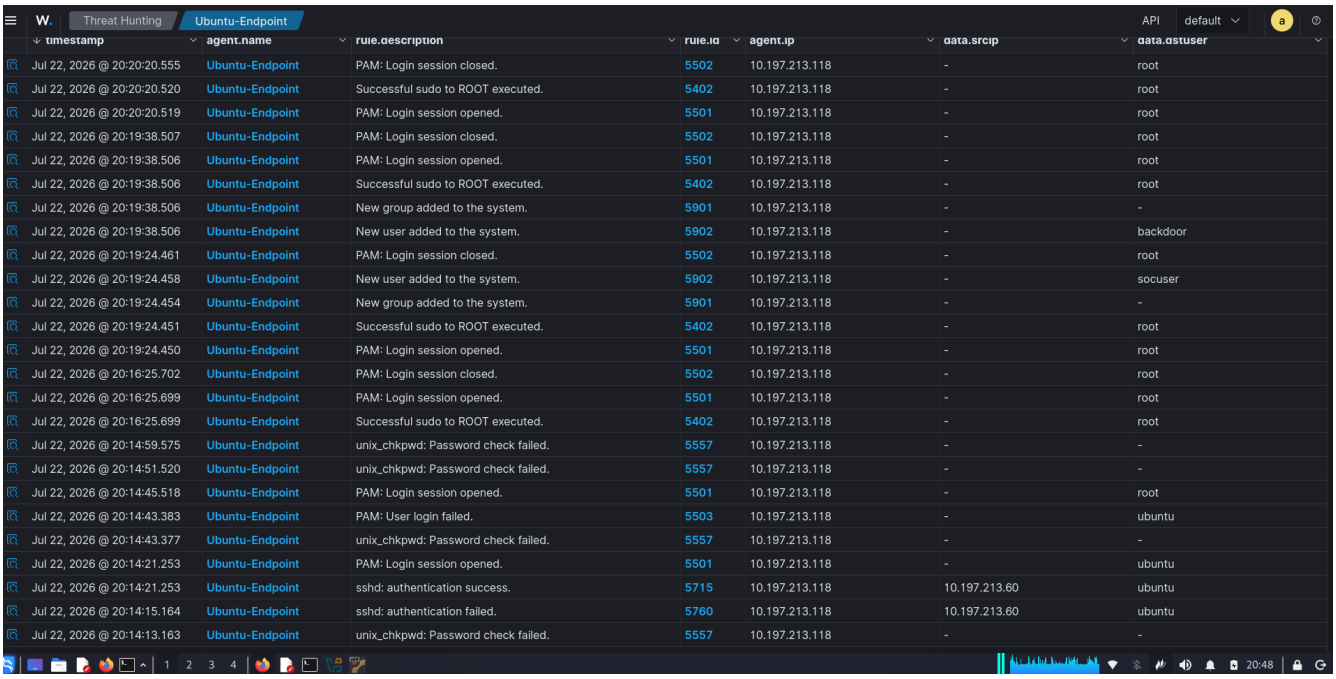


Figure 8: Ubuntu authentication and system events displayed in the Wazuh Dashboard.



timestamp	agent.name	rule.description	rule.id	agent.ip	data.srcip	data.dstuser
Jul 22, 2026 @ 20:20:20.555	Ubuntu-Endpoint	PAM: Login session closed.	5502	10.197.213.118	-	root
Jul 22, 2026 @ 20:20:20.520	Ubuntu-Endpoint	Successful sudo to ROOT executed.	5402	10.197.213.118	-	root
Jul 22, 2026 @ 20:20:20.519	Ubuntu-Endpoint	PAM: Login session opened.	5501	10.197.213.118	-	root
Jul 22, 2026 @ 20:19:38.507	Ubuntu-Endpoint	PAM: Login session closed.	5502	10.197.213.118	-	root
Jul 22, 2026 @ 20:19:38.506	Ubuntu-Endpoint	PAM: Login session opened.	5501	10.197.213.118	-	root
Jul 22, 2026 @ 20:19:38.506	Ubuntu-Endpoint	Successful sudo to ROOT executed.	5402	10.197.213.118	-	root
Jul 22, 2026 @ 20:19:38.506	Ubuntu-Endpoint	New group added to the system.	5901	10.197.213.118	-	-
Jul 22, 2026 @ 20:19:38.506	Ubuntu-Endpoint	New user added to the system.	5902	10.197.213.118	-	backdoor
Jul 22, 2026 @ 20:19:24.461	Ubuntu-Endpoint	PAM: Login session closed.	5502	10.197.213.118	-	root
Jul 22, 2026 @ 20:19:24.458	Ubuntu-Endpoint	New user added to the system.	5902	10.197.213.118	-	socuser
Jul 22, 2026 @ 20:19:24.454	Ubuntu-Endpoint	New group added to the system.	5901	10.197.213.118	-	-
Jul 22, 2026 @ 20:19:24.451	Ubuntu-Endpoint	Successful sudo to ROOT executed.	5402	10.197.213.118	-	root
Jul 22, 2026 @ 20:19:24.450	Ubuntu-Endpoint	PAM: Login session opened.	5501	10.197.213.118	-	root
Jul 22, 2026 @ 20:16:25.702	Ubuntu-Endpoint	PAM: Login session closed.	5502	10.197.213.118	-	root
Jul 22, 2026 @ 20:16:25.699	Ubuntu-Endpoint	PAM: Login session opened.	5501	10.197.213.118	-	root
Jul 22, 2026 @ 20:16:25.699	Ubuntu-Endpoint	Successful sudo to ROOT executed.	5402	10.197.213.118	-	root
Jul 22, 2026 @ 20:14:59.575	Ubuntu-Endpoint	unix_chkpwd: Password check failed.	5557	10.197.213.118	-	-
Jul 22, 2026 @ 20:14:51.520	Ubuntu-Endpoint	unix_chkpwd: Password check failed.	5557	10.197.213.118	-	-
Jul 22, 2026 @ 20:14:45.518	Ubuntu-Endpoint	PAM: Login session opened.	5501	10.197.213.118	-	root
Jul 22, 2026 @ 20:14:43.383	Ubuntu-Endpoint	PAM: User login failed.	5503	10.197.213.118	-	ubuntu
Jul 22, 2026 @ 20:14:43.377	Ubuntu-Endpoint	unix_chkpwd: Password check failed.	5557	10.197.213.118	-	-
Jul 22, 2026 @ 20:14:21.253	Ubuntu-Endpoint	PAM: Login session opened.	5501	10.197.213.118	-	ubuntu
Jul 22, 2026 @ 20:14:21.253	Ubuntu-Endpoint	sshd: authentication success.	5715	10.197.213.118	10.197.213.60	ubuntu
Jul 22, 2026 @ 20:14:15.164	Ubuntu-Endpoint	sshd: authentication failed.	5760	10.197.213.118	10.197.213.60	ubuntu
Jul 22, 2026 @ 20:14:13.163	Ubuntu-Endpoint	unix_chkpwd: Password check failed.	5557	10.197.213.118	-	-

Figure 9: Ubuntu authentication and system events displayed in the Wazuh Dashboard.

Ubuntu Log Sources

Log Source	Description	Onboarding Method
Authentication Logs (`auth.log`)	Records SSH logins, failed authentication attempts, and `sudo` activity.	Monitored automatically by the Wazuh Agent.
System Log (`syslog`)	Records general operating system events and services.	Collected through the Wazuh Agent.
SSH Events	Records successful and failed remote login attempts.	Captured from the authentication logs and forwarded by the Wazuh Agent.
Package Management (`apt`)	Records software installation and update activities.	Logged by the operating system and collected through system logs.

pfSense Firewall Log Integration

Once I had successfully onboarded the Windows 11 and Ubuntu endpoints into Wazuh, the next logical step was to bring the network firewall on board as another log source. By

collecting firewall events, I could keep an eye on network traffic alongside the endpoint activity right from the console.

To kick things off, I needed to configure pfSense to send its logs to the Kali Linux machine that was running the Wazuh Manager. In the pfSense web console, I headed over to the Remote Logging settings to enable Syslog forwarding. To ensure the firewall events were transmitted, I set the Destination to the IP address of the Kali host, using UDP port 514.

With pfSense now forwarding its logs, I moved on to set up Kali Linux's rsyslog to accept those Syslog messages. Instead of sending the logs directly to Wazuh, rsyslog would first capture them and write them to a log file located at /var/log/pfsense.log. As this file got updated, it confirmed that new firewall events were being logged for every network packet that passed through pfSense.

Even though the firewall logs were successfully reaching Kali Linux, they still weren't showing up in Wazuh. This highlighted an important point: accepting Syslog messages and ingesting logs into Wazuh are two distinct processes. So here is the solution for this, I edited the wazuh manager configuration and configured /var/log/pfsense.log as a monitored log source in the <localfile> section of the ossec.conf file. Starting the Wazuh Manager, the logcollector service was enabled to be automatically monitoring the pfSense log file.

The next challenge was to make sure that Wazuh was able to analyze the events from the firewall. My first idea was to build my own decoder that was able to strip out the IP address of the source, destination, protocol, ports, interface and firewall action. But when testing the custom decoder, however, errors were made during the configuration than prevented Wazuh Manager from starting. Upon further investigation and after testing with wazuh-logtest, I realized that Wazuh v 4.14.6 is already equipped with a pfSense filterlog event decoder. It resulted in the custom decoder being removed and Wazuh to be configured to use its own parser.

The built-in decoder was used, and firewall events were successfully decoded into structured fields, such as source IP, target IP, protocol, ports, packet length and firewall action (allow or block). A local rule was then defined to create a unique alert for every pfSense firewall event that was processed so it would be easier to find them while investigating events.

I created some network traffic both from the Windows 11 VM and Ubuntu VM to check for proper integration. Less than a few minutes later, firewall alerts started popping up in the Wazuh Threat Hunting Event dashboard. All the logs indicated communication

between the Ubuntu endpoint (192.168.56.104), the pfSense firewall (192.168.56.106), and external addresses. When Wazuh was opened an alert, the following key network properties were identified by Wazuh: Source IP, Destination IP, Protocol, Ports, and Firewall Action.

With the addition of pfSense, you now have all of the SIEM log sources in the SIEM lab. The logs of Windows, Ubuntu and the network firewall are now received and correlated to the Wazuh server, giving centralized visibility of endpoint and network activity.

Tools used

Component	**Purpose**
pfSense	Firewall and network traffic filtering
rsyslog	Receives Syslog messages on Kali Linux
Wazuh Logcollector	Reads the pfSense log file
Wazuh Manager	Decodes and analyzes firewall events
Wazuh Dashboard	Displays firewall alerts

Findings

SIEM Dashboard Development

This phase of the SIEM lab involved creating custom dashboards within the Wazuh Dashboard to provide real-time visibility into authentication, endpoint, and network activity. Each dashboard was populated with live telemetry collected from the Windows 11 endpoint through the Wazuh agent and Sysmon, ensuring that all visualizations represented actual events generated during testing rather than sample or demonstration data.

Dashboard 1 – Authentication Activity Dashboard

Objective

The goal here is to keep an eye on Windows authentication activity by differentiating between successful and failed login attempts, while also pinpointing the hosts and user accounts involved. What visualizations were created in my dashboard:

- A line chart showing login successes and failures over time

- A bar chart illustrating authentication activity by host
- A vertical bar chart detailing authentication activity by user account
- A data table listing the top accounts with failed logins.

The dashboard effectively showcased the authentication events that were generated during user login testing. We intentionally executed multiple failed logins to ensure event collection was working properly, which allowed us to link those failed authentication attempts to specific hosts and user accounts.

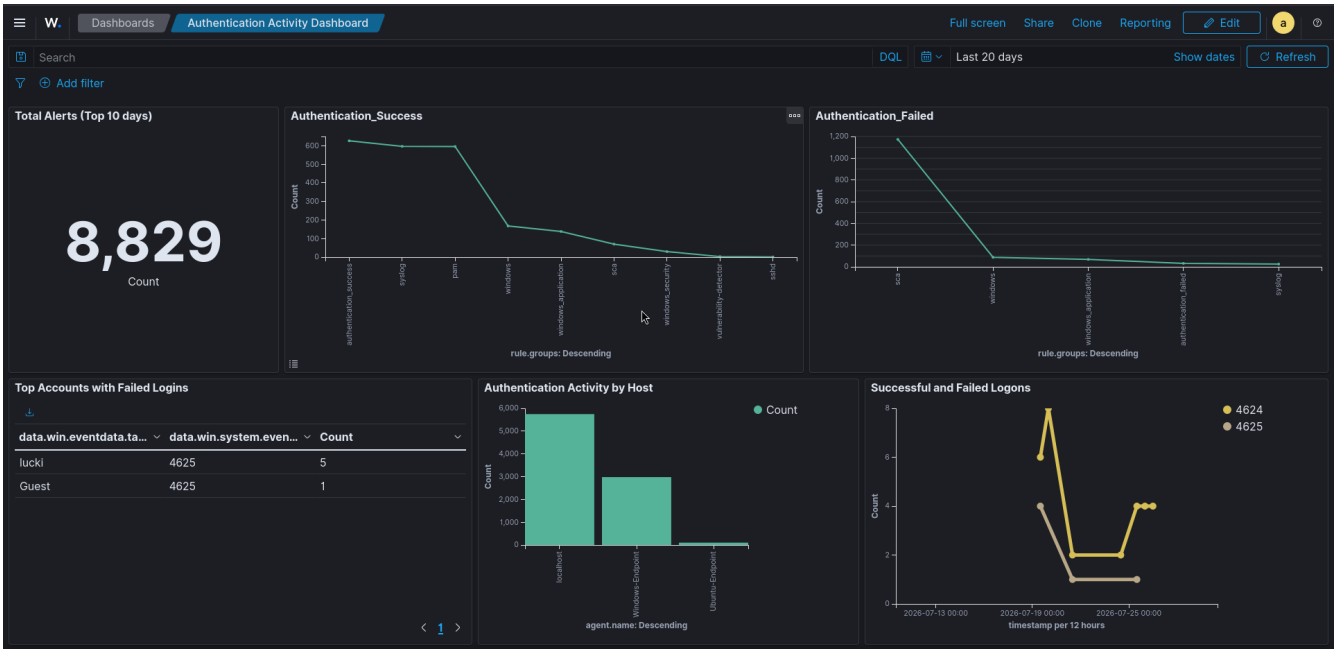


Figure 10: Authentication Activity Dashboard

Dashboard 2 – Endpoint Process Activity Dashboard

Objective

To monitor process execution on the Windows endpoint using Symon Process Creation (Event ID 1), providing visibility into endpoint activity and parent-child process relationships. What dashboards were created:

- Most frequently executed processes
- Parent-child process relationship table
- Recent process creation events feed

The dashboard provided visibility into processes launched during normal system activity and testing. Applications such as PowerShell, Command Prompt, Notepad, browsers, and system utilities generated live Sysmon events that were visualized in real time.

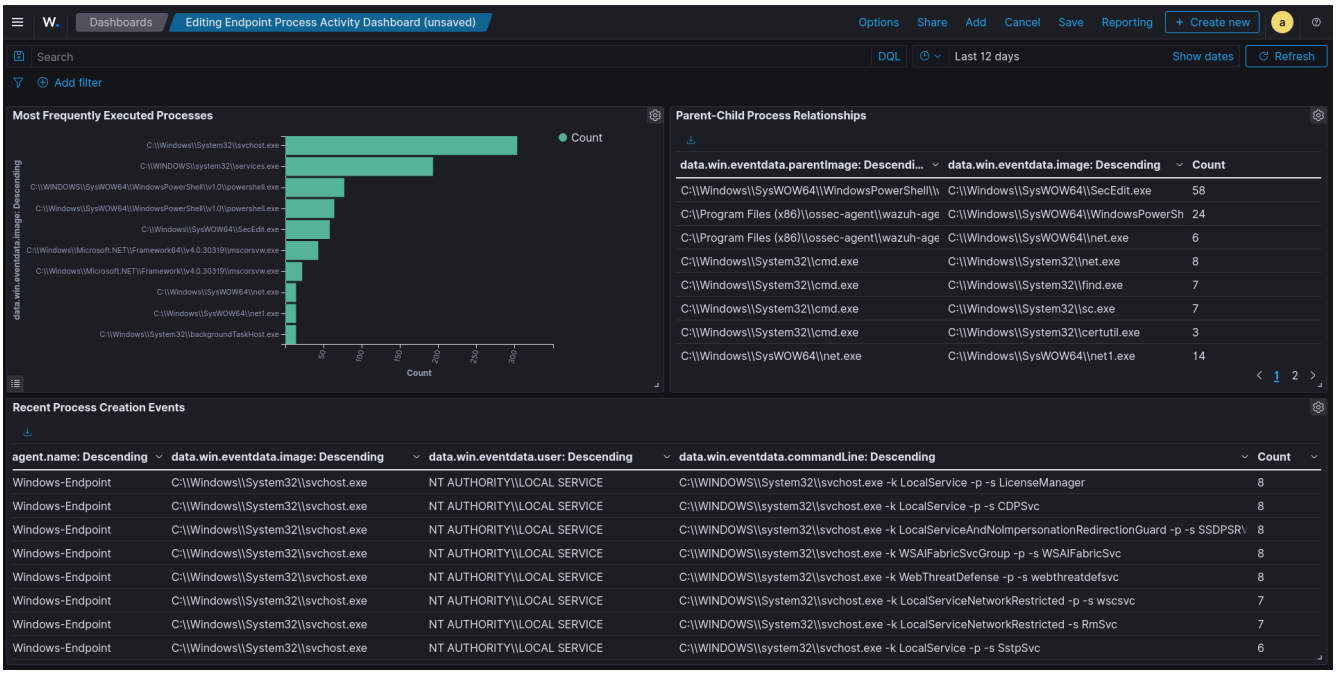


Figure 11: Endpoint Process Activity Dashboard

Dashboard 3 – Network Traffic Overview Dashboard

Objective

To monitor outbound network connections originating from the Windows endpoint and identify frequently contacted systems.

Visualizations Created:

- Traffic volume over time
- Top source IP addresses
- Top destination IP addresses
- Recent network connection events

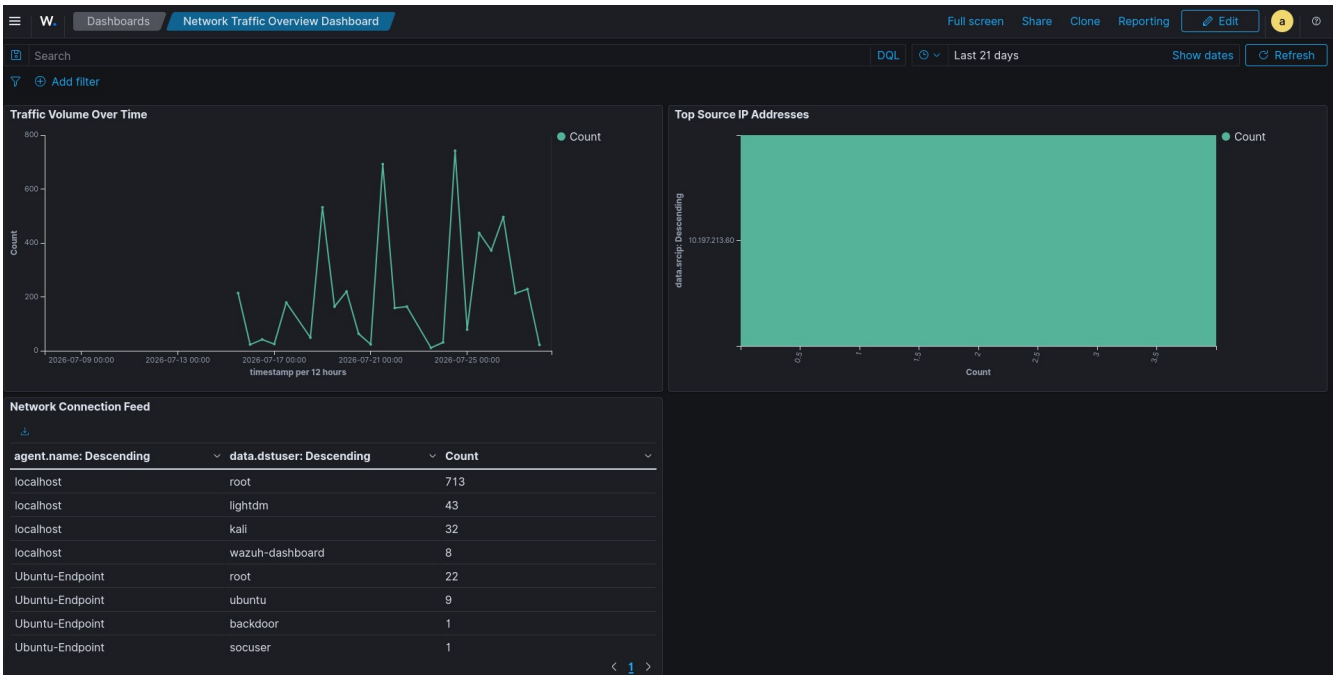


Figure 12: Network Traffic Overview Dashboard

Correlation Rules

After successfully collecting and visualizing the logs from Windows endpoint, Ubuntu endpoint and pfSense firewall successfully, I have defined custom correlation rules to better detect suspicious activities with Wazuh. The goal was to show how the SIEM could be used to find security events that would not be seen by single logs.

For the detection, two custom detection rules were chosen:

- Failed Login Threshold (Brute Force Detection) Rule 1 keeps you from getting too many failed login attempts from another user.
- Detect suspicious process usage in PowerShell

Every rule was designed with attack data and checked for false positives.

Rule 1 – Failed Login Threshold

The rule monitors authentication failure events generated by Windows Security Logs and Linux authentication logs.

An alert is generated when:

- Five failed login attempts
- Occur within five minutes

- Against the same username

This behavior is commonly associated with password guessing or brute-force attacks.

```
<group name="custom,authentication">

  <rule id="100100" level="10" frequency="5" timeframe="300">

    <if_matched_sid>5710</if_matched_sid>

    <same_user />

    <description>
Possible Brute Force Attack - 5 failed logins within 5 minutes.
    </description>

    <mitre>
      <id>T1110</id>
    </mitre>

    <group>
authentication,bruteforce,
    </group>

  </rule>

</group>
```

Figure 13: Rule for Possilbe Brute Force Attack

This rule was designed to find a possible brute force attack on user accounts. This rule does not trigger an alarm per failed authentication attempt, but rather it will trigger alarms if a failed login is repeated within a short time period from the same account. This method is a huge help in minimizing alert fatigue and pinpointing potentially malicious activity.

Rule 2 – Suspicious PowerShell Execution

PowerShell is among the most misused administration tools in attacks towards Windows. PowerShell is often employed in attacks to run malicious scripts, download malicious files, conduct reconnaissance, or achieve persistence.

This rule is used to identify suspicious PowerShell activity from uncommon execution paths or command-line arguments that are often used in malicious activity.

The rule is for Sysmon process creation events on the Windows endpoint.

The rule fires an alert if PowerShell runs with the following signs of suspicious activity:

- Encoded commands
- Hidden execution
- Bypass execution policy
- Suspicious command-line arguments

These execution patterns are not used very much in normal administration.

```
<group name="custom,process">
<rule id="100101" level="12">
<if_group>sysmon_event1</if_group>
<field name="win.eventdata.parentImage">powershell.exe</field>
<field name="win.eventdata.image">cmd.exe</field>
<description>
PowerShell spawned cmd.exe.
Possible suspicious process chain.
</description>
<mitre>
<id>T1059</id>
</mitre>
<group>
process_creation,powershell,
</group>
</rule>
</group>
```

Figure 14: Rule for Suspicious Powershell Process

Gaps and Challenges

There were some difficulties encountered during the implementation of this SIEM laboratory in integrating all the logs sources into Wazuh. I had the chance to learn more about log collection, log parsing, and troubleshooting in a SIEM environment in each issue. The most difficult log to onboard was the pfSense firewall. In the case of the firewall, it's not necessary to install and configure the Wazuh agent as it is on Windows or Ubuntu endpoints, only forward the Syslog messages to the Wazuh manager. The logs were received and written in the Kali Linux system in /var/log/pfsense.log but did not show up in the Wazuh dashboard right away.

To resolve the problem, the following steps were taken: Syslog service validation, modification of ossec.conf to include the monitoring of the firewall log file, validation of wazuh configuration files, firewall events verification using wazuh-logtest and checking that the events coming from the firewall were being indexed. It took several configuration issues to experiment with custom decoders and realize that Wazuh already has an integrated decoder that could decode pfSense filterlog events. After getting rid of the superfluous custom decoder, firewall events were successfully processed and appeared in the dashboard. The other challenge was creating meaningful dashboards. The logs from various operating systems are stored in different field-structures in Wazuh and the correct field-structures to be used in visualizations had to be carefully inspected.

The process included trying several field combination in the field, then generating dashboards and validating their accuracy with regards to authentications, process creation, and network traffic. Despite these problems, every problem was addressed through systematic troubleshooting, log validation and configuration testing.

Although the completed SIEM lab provides visibility into Windows, Linux, and firewall events, several important data sources are still absent.

Recommendations and Next Steps

The set up SIEM lab is a proof of concept that allows to implement centralized log collection, event correlation, dashboard generation and alerts with Wazuh. There are a few adjustments that would further benefit the use of the lab, as well as my hands-on SOC experience.

I would first want to add a network intrusion detection system (NIDS) like Suricata to pfSense given more time. This would enable the SIEM to get IDS alerts, along with firewall logs, giving the SIEM much greater visibility of network attacks and suspicious traffic.

I would also add more custom correlation rules for other scenarios like privilege escalation, persistence, suspicious process execution, lateral movement, etc. More custom detections would help to simulate the work done by a Security Operations Center (SOC) analyst.

Another enhancement could be the acquisition of DNS logs and Web proxy logs. These logs would give valuable information about browsing habits, outbound, and interactions with potentially malicious domains, thus the dashboards would give a more comprehensive picture of network activity.

I would add more dashboards and reports for file integrity monitoring, firewall threat trends, endpoint security events and system health to further enhance the investigation process. These dashboards would assist analysts in detecting abnormal behaviour in an investigation faster.

Lastly, I would be connecting a threat intelligence platform like Virus Total or using a threat feed from outside sources for automatic correlation of observed IP addresses, domains and file hashes with known bad ones. This will provide contextual intelligence to alerts, and enhance incident response efficiency.

In summary this project helped to lay the groundwork for an effective SIEM environment and identified a number of areas in which it could be expanded. With the addition of more data sources, advanced detections, and external threat intelligence, the laboratory could closely resemble the capabilities of an enterprise Security Operations Center.

This project proved to be a success for deploying a working SIEM laboratory with Wazuh. Windows 11, Ubuntu Linux, and pfSense were then connected to a centralized monitoring platform that could gather, analyze and visualize security events. Custom dashboards and correlation rules were created for better visibility of authentication, endpoint and network activity. In general, the project offered hands-on experience with SIEM deployment, log management, event correlation, and security monitoring, which closely mimicked the processes in a real SOC.